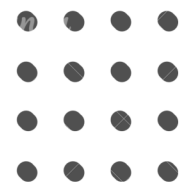




VMES Grinds





**Leetcode: 187. Repeated DNA
Sequences
Difficulty – Medium**





187. Repeated DNA Sequences

Solved

Medium

Topics

Companies

The **DNA sequence** is composed of a series of nucleotides abbreviated as 'A', 'C', 'G', and 'T'.

- For example, "ACGAATTCCG" is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the **10-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.



Example 1:

Input: `s = "AAAAACCCCCAAAAACCCCCAAAAAGGGTTT"`

Output: `["AAAAACCCCC", "CCCCCAAAA"]`

Example 2:

Input: `s = "AAAAAAAAAAAA"`

Output: `["AAAAAAAAAA"]`



Constraints:

- `1 <= s.length <= 105`
- `s[i]` is either `'A'`, `'C'`, `'G'`, or `'T'`.



Brain Storm 3 mins





Intuition





Repeated DNA Sequences

1. **Store, Count, Look up** – ok – I will use hashmap
2. **Get the substrings** – ok – how can i get it?
 - a. **10 chars** – how will i loop it? `range(0, len(s)-9, 10)`
 - b. **what should i consider** – list index out of bound
 - c. **ok i got the substring** → store it in `hashMap`
 - d. **how to store** → key? value?
 - e. **key** → substring
 - f. **value** → count
3. **After storing all the substrings**
 - a. **if count > 1** → append to list





Explanation





Repeated DNA Sequences

AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT

Starts from where?, increment by?, ends -where?

Stores the substrings with their counts

Check the condition



Code





Very First Submission

```
class Solution:
    def findRepeatedDnaSequences(self, s: str) -> List[str]:
        j = 10
        hashMap = {}
        answer = []
        for i in range(len(s)):
            print(i, i+j, len(s))
            if i+j <= len(s):
                hashMap[s[i:(i+j)]] = hashMap.get(s[i:(i+j)], 0) +1
        print(hashMap)
        for key,value in hashMap.items():
            if value > 1:
                answer.append(key)
        return answer
```

⤴ View less



Journey to the optimised solution



VMES Grinds

Then – finding out how can I simplify it

Thinking other data structures I can use



Optimised Code





```
1 class Solution:
2     def findRepeatedDnaSequences(self, s: str) -> List[str]:
3         hashSet = set()
4         answer = set()
5         for i in range(len(s) - 9):
6             seq = s[i:(i+10)]
7             if seq not in hashSet:
8                 hashSet.add(seq)
9             else:
0                 answer.add(seq)
1         return list(answer)
```



Time and Space Complexity Analysis





```
1 class Solution:
2     def findRepeatedDnaSequences(self, s: str) -> List[str]:
3         hashSet = set()
4         answer = set()
5         for i in range(len(s) - 9):
6             seq = s[i:(i+10)]
7             if seq not in hashSet:
8                 hashSet.add(seq)
9             else:
0                 answer.add(seq)
1         return list(answer)
```

Time Complexity - $O(n)$

Space - $O(n)$



Key Takeaway



Takeaway 1: Remember to consider other data
structure you can use as well